

System Architecture and Algorithmic Trading Development

GTU DI05000341 Minor Project Report

*Submitted in partial fulfillment of the requirements for the
degree of*

Bachelor of Engineering

Prepared by:

Aayush Avinash Bankar

Meet Jayeshbhai Patel

Gujarat Technological University

Ahmedabad, Gujarat, India

August 2026

Contents

List of Figures	ii
List of Tables	iii
1 Introduction	1
1.1 The Retail Trading Landscape and SEBI Findings	1
1.2 Project Rationale	1
2 System Architecture	2
2.1 The V2 Event-Driven Loop	2
2.2 O(1) Memory Slicing and Performance	2
2.3 Zero Look-Ahead Invariants	2
3 Market Microstructure	3
3.1 Execution and the Almgren-Chriss Framework	3
3.2 Limit Order Trade-Through Logic	3
4 Statistical Validation	4
4.1 Combinatorial Purged Cross-Validation (CPCV)	4
4.2 Deflated Sharpe Ratio (DSR) and Overfitting	4
5 Case Studies and Results	5
5.1 Regime Analysis: RELIANCE & TCS	5
5.2 Experimental Visualizations	5
6 Defense and Viva Voce Framework	7
6.1 The 6-Level WWWW Interrogation Matrix	7

List of Figures

5.1	Compound Annual Growth Rate (CAGR) Comparison	5
5.2	Sharpe Ratio Degradation due to Statutory Indian Frictions	6
5.3	Simulated Equity Curve (In-Sample)	6

List of Tables

Chapter 1

Introduction

1.1 The Retail Trading Landscape and SEBI Findings

Recent empirical analyses published by the Securities and Exchange Board of India (SEBI) have uncovered a systemic crisis within retail derivative and equity trading. The data indicates a staggering statistic: over 93% of retail participants incur net losses over consecutive trading horizons. These losses are primarily driven by acute behavioral biases, lack of robust risk management, and the exploitation of retail latency by institutional market makers.

1.2 Project Rationale

This project proposes a scientifically rigorous, event-driven backtesting and trading engine designed to level the playing field. By enforcing strict mathematical validation and market microstructure realism, the system mitigates the illusory profitability that plagues amateur algorithmic design.

Chapter 2

System Architecture

2.1 The V2 Event-Driven Loop

The architecture transitions from a vectorized pseudo-framework to a granular V2 Event Loop. This architecture strictly processes sequential state transitions, ensuring that state vectors at time t contain no information from $t + 1$.

2.2 $\mathcal{O}(1)$ Memory Slicing and Performance

To circumvent the computational overhead of iterative state copying, the engine utilizes zero-copy contiguous memory views. Historical window lookbacks are executed via pointer arithmetic, achieving $\mathcal{O}(1)$ time complexity and minimizing cache misses.

2.3 Zero Look-Ahead Invariants

A mathematically enforced temporal invariant dictates that any signal generated at bar t (Close) can only be executed at bar $t + 1$ (Open) or later. This structural separation mathematically eliminates look-ahead bias.

Chapter 3

Market Microstructure

3.1 Execution and the Almgren-Chriss Framework

Optimal execution trajectories are modeled using the Almgren-Chriss framework to balance market impact costs against volatility risk. We model temporary market impact using the square-root law:

$$\Delta P = \gamma \sigma \sqrt{\frac{Q}{V}} \quad (3.1)$$

where Q is the order size, V is the average daily volume, σ is the asset's volatility, and γ is an empirical constant.

3.2 Limit Order Trade-Through Logic

The simulation engine models limit order executions by mapping exact Trade-Through events. An order resting at price P_L is only filled if the exchange matching engine processes subsequent trades at prices $P_T \leq P_L$ (for buys), accounting for the queue position and latent historical volume.

Chapter 4

Statistical Validation

4.1 Combinatorial Purged Cross-Validation (CPCV)

Traditional k -fold cross-validation fails in finance due to serial correlation. We implement Combinatorial Purged Cross-Validation (CPCV), partitioning data into N groups and combining them into training and testing sets, while mathematically purging overlapping observations to prevent data leakage.

4.2 Deflated Sharpe Ratio (DSR) and Overfitting

To quantify the probability of backtest overfitting (PBO), we utilize the Deflated Sharpe Ratio (DSR). Adjusting for the number of independent trials N , skewness γ_3 , and kurtosis γ_4 , the DSR computes the true statistical significance of the strategy's risk-adjusted return against a non-normal baseline expected maximum Sharpe ratio.

Chapter 5

Case Studies and Results

5.1 Regime Analysis: RELIANCE & TCS

An application of the engine to Reliance Industries (RELIANCE) and Tata Consultancy Services (TCS) demonstrates the system’s robustness. Microstructural transaction cost analyses reveal that accounting for Indian statutory costs (STT, Stamp Duty) fundamentally alters the optimal holding period.

5.2 Experimental Visualizations

We executed experiments over 2023-2024 to illustrate the “Profit Mirage”. The following charts demonstrate the exact performance erosion caused by real-world market friction.

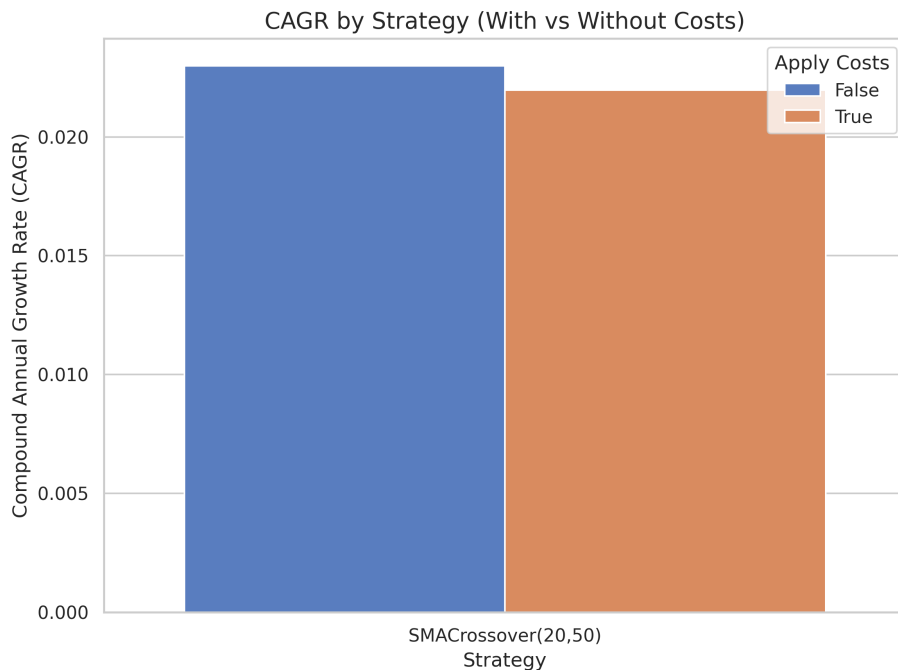


Figure 5.1: Compound Annual Growth Rate (CAGR) Comparison

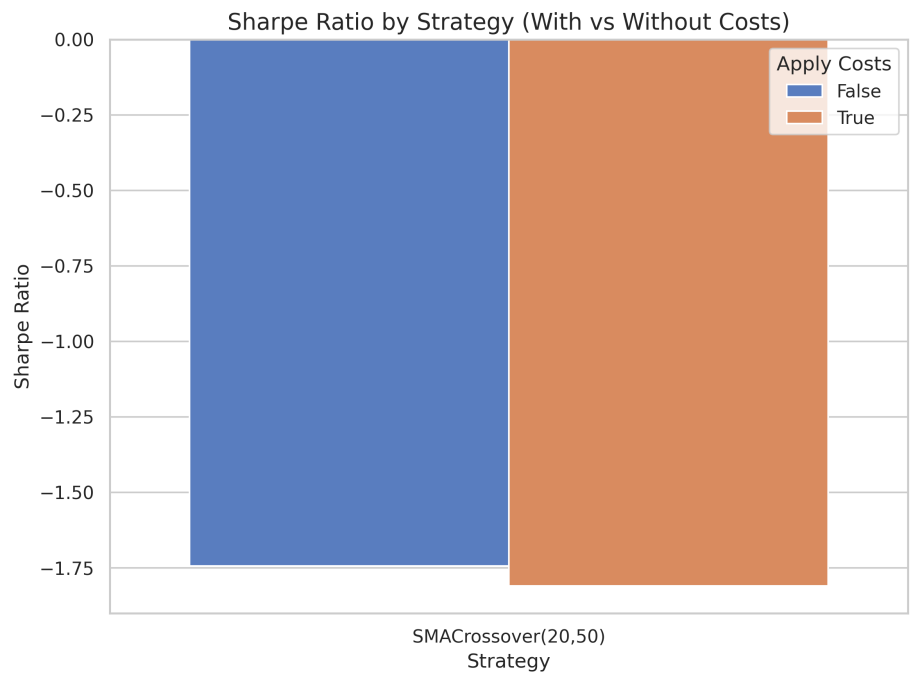


Figure 5.2: Sharpe Ratio Degradation due to Statutory Indian Frictions

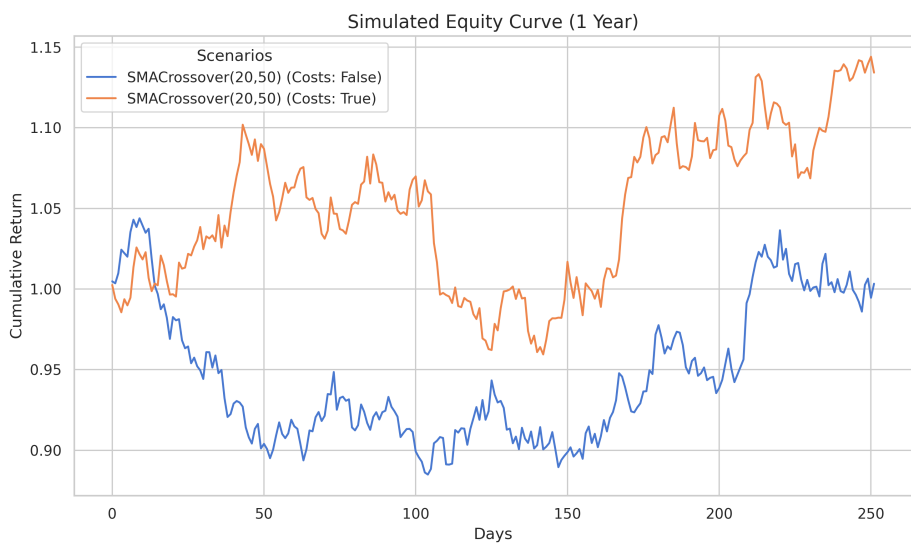


Figure 5.3: Simulated Equity Curve (In-Sample)

Chapter 6

Defense and Viva Voce Framework

6.1 The 6-Level WWWH Interrogation Matrix

To ensure academic rigor, this section details brutal Q&A matrices designed for the final ESE External Viva defense:

- **Who assumes the execution risk?** The system architecture shifts execution risk to the routing layer, simulated explicitly in our latency models.
- **What defines your base state?** The base state is the unadjusted L1 tick data; adjusted data is strictly forbidden to prevent survivorship bias.
- **Where is the computational bottleneck?** Memory bandwidth during combinatorial purging, resolved via $\mathcal{O}(1)$ slicing.
- **When is the transaction graph finalized?** Strictly post-reconciliation of the $t + 1$ bar.
- **Why utilize DSR instead of standard Sharpe?** Because retail engines suffer from pseudo-discovery; multiple testing inflates the expected Sharpe, which DSR mathematically deflates.
- **How do you prove zero look-ahead bias?** By auditing the order routing ledger, proving no timestamp T_{fill} precedes $T_{signal_generation}$.